Size-Scaled AlphaZero Self-Play for Hex: A Single-Network Agent Across Board Sizes 5 to 17

Chorn Philipp **Hromada Stefan** **Wang Li Wen**

Cahya Wirawan
University of Applied Sciences Technikum Wien (FHTW)
{ai25m0xx, ai25m0xx, ai25m0xx, ai25m063}@technikum-wien.at

Abstract

Mastering the board game Hex with self-play reinforcement learning is challenging because the branching factor and the length of the shortest winning connection both grow quadratically with board side length, so a recipe tuned for a small board collapses into near-random play on a larger one. We present a self-contained AlphaZero-style agent for Hex that addresses this through a single design principle: every search and learning quantity is derived automatically from the board size. The simulation budget, residual-network width and depth, self-play volume, replay capacity, and mini-batch size are all scaled by closed-form rules so that Monte Carlo Tree Search visits each root child enough times for its visit-count distribution to carry a usable policy target. A colour-agnostic perspective frame lets one network play both players, and a compiled C++ tree-search kernel with batched, parallel leaf evaluation keeps the GPU saturated. We further integrate five engineering refinements common to strong implementations: rhombus symmetry augmentation, sub-tree reuse across moves, calibrated resignation, mixed-precision training, and decoupled weight decay with a cosine schedule. Trained from scratch on a single GPU, the agent reaches a 90–100% win rate against a uniform-random opponent on 11×11 and 13×13 boards within roughly 40 self-play iterations, and continues to improve on 15×15. We describe the scaling rules, the search engine, and the training loop in full, and discuss what is needed to move from defeating random play to matching specialist Hex engines.

1 Introduction

Hex is a two-player connection game invented independently by Piet Hein and John Nash. Players alternately place stones on the cells of an $n \times n$ rhombus; one player wins by forming an unbroken chain of their colour between their two opposing edges. The game is elegant for theory and demanding for practice: it can never end in a draw, the first player provably has a winning strategy by a strategy-stealing argument (**?**), yet that strategy is unknown for all but the smallest boards. Hex has therefore long served as a benchmark for game-playing artificial intelligence (**??**), and it is the basis of the reinforcement learning assignment at FHTW for which this work was developed: students implement an agent behind a fixed engine interface and play it against the provided environment.

The dominant paradigm for games of this kind is AlphaZero (**?**), which couples Monte Carlo Tree Search (MCTS) with a deep neural network trained purely on self-play. The network supplies move priors and position values to guide the search, while the visit counts the search produces become a stronger policy target than the raw network output. This mutual reinforcement—search improving the targets, the network improving the search—is what drives learning without any human games or heuristics. The recipe is conceptually simple but notoriously sensitive: too few simulations and the

Preprint. Under review.

visit-count policy target is dominated by noise, the bootstrap never ignites, and the agent plateaus at random play.

This sensitivity is sharpened on Hex because difficulty scales steeply with the board. The root branching factor is n^2, and the minimum number of stones needed to span the board grows linearly in n, so both the breadth and the depth of meaningful search grow with size. A fixed simulation count that is generous on a 5×5 board spreads only a fraction of a visit across each child on a 15×15 board, where the root has 225 legal moves. A practitioner who hard-codes hyperparameters tuned for a small board and simply increases n will watch the agent fail to learn, with no error to diagnose.

We address this with a deliberately small idea applied consistently: *let the board size determine everything*. From a single integer n we derive the per-move simulation budget, the network's channel count and residual depth, the number of self-play games per iteration, the replay-buffer capacity, and the mini-batch size, each from a closed-form rule motivated by how that quantity must grow to keep learning healthy. Around this core we build a complete, reproducible system: a colour-agnostic perspective frame so one network plays both sides, a compiled C++ search kernel with virtual-loss parallelism and batched leaf evaluation, and five refinements drawn from the modern AlphaZero literature.

In summary, our contributions are:

- We derive and justify a set of *board-size scaling rules* that jointly set the MCTS budget, network capacity, and data volume, and we explain why each must grow with n for the self-improvement loop to bootstrap.
- We describe a complete, single-GPU AlphaZero implementation for Hex featuring a perspective-frame single network, a compiled C++ MCTS engine with parallel virtual-loss leaf collection, and pipelined multi-process self-play.
- We integrate and document five practical refinements—symmetry augmentation, sub-tree reuse, calibrated resignation, mixed precision, and AdamW with cosine annealing—and report empirical learning curves across board sizes 11×11 through 15×15.

The remainder of the paper reviews related work (Section **??**), details the method and scaling rules (Section **??**), describes the experimental setup (Section **??**), reports results (Section **??**), interprets them (Section **??**), and concludes (Section **??**).

2 Related Work

2.1 Self-play search-and-learn agents

AlphaGo first combined supervised policy learning, MCTS, and reinforcement learning to defeat a professional Go player (**?**). AlphaGo Zero removed human data, learning entirely from self-play with a single residual network producing both policy and value (**?**), and AlphaZero generalised the method to chess and shogi (**?**). MuZero further removed the need for a known environment model by learning a latent dynamics model jointly with the value and policy (**?**). Our work is a faithful instance of the AlphaZero template rather than a new algorithm; our concern is the engineering and hyperparameter discipline needed to make that template learn reliably on Hex across a range of board sizes on modest hardware.

2.2 Reinforcement learning for Hex

Hex has a rich history as an AI testbed. Expert Iteration (ExIt) was introduced and evaluated on Hex, framing the search-and-learn loop as an imitation of a slow expert by a fast apprentice (**?**). Deep convolutional networks were shown to predict expert Hex moves and to strengthen the strong solver-based engine MoHex (**?**), whose lineage rests on the connection-based theory and inferior-cell analysis catalogued by **?**. These specialist systems combine learned evaluation with Hex-specific knowledge such as virtual connections and dead-cell pruning. Our agent deliberately uses *no* Hex-specific knowledge beyond the win condition and a board symmetry; it learns connectivity implicitly through the value head. This makes it weaker than knowledge-rich engines but cleanly isolates what a generic AlphaZero pipeline can achieve, and clarifies the gap that domain knowledge would later fill.

2.3 Monte Carlo Tree Search and its parallelisation

MCTS with the UCT selection rule (**?**) and the broader family surveyed by **?** underpins modern game agents. AlphaZero replaces UCT's exploration term with PUCT, in which a learned prior biases exploration toward promising moves. Parallelising MCTS to feed a GPU efficiently requires care: virtual loss (**?**) temporarily penalises in-flight nodes so concurrent simulations diverge to different branches, enabling many leaves to be evaluated in one batched network call. Large-scale systems such as ELF OpenGo (**?**) and KataGo (**?**) demonstrate how batching, sub-tree reuse, and playout-cap or resignation tricks raise self-play throughput by orders of magnitude. KataGo in particular contributes efficiency techniques—auxiliary targets, playout caps, and aggressive batching—several of which inform the refinements we adopt.

2.4 Value-based and policy-gradient baselines

Before self-play search, the standard reinforcement learning toolkit comprises tabular Q-learning (**?**), Deep Q-Networks (**?**), the REINFORCE policy gradient (**?**), and Proximal Policy Optimization (**?**). These methods learn a direct mapping from state to action or value without lookahead at decision time. On a game with Hex's combinatorial depth they struggle to assign credit through long connection sequences, which is precisely the weakness that decision-time search remedies. Our repository includes these methods as baselines, and their comparative weakness is the empirical motivation for the search-based agent that is the focus of this paper.

Research gap. The AlphaZero recipe is well documented for fixed, large problems with industrial compute, and Hex-specific engines lean heavily on domain knowledge. What is comparatively under-documented is how to make a *knowledge-free* AlphaZero agent learn *robustly across a range of board sizes* on a single GPU, where the same code must succeed on a 7×7 board and a 17×17 board without manual retuning. This paper addresses that gap with explicit, justified size-scaling rules and a complete reproducible implementation.

3 Method

3.1 Game representation and the perspective frame

The engine represents the board as an $n \times n$ integer grid with values $\{+1, -1, 0\}$ for the two players and empty. Red connects left to right, Blue connects top to bottom. Rather than train separate networks or feed the side-to-move as an extra plane, we adopt a *perspective frame*: when it is Blue's turn, the board is recoded by transposing along the short diagonal and swapping colours, so that from the network's point of view the player to move is always "Red, connecting left to right." A chosen move is mapped back to true coordinates by the same self-inverse transform. Every stored state is thus in a canonical frame, which both halves the function the network must learn and makes data augmentation straightforward.

The network input is a $2 \times n \times n$ tensor whose two channels hold, respectively, the stones of the player to move and the stones of the opponent in the perspective frame.

3.2 Network architecture

The agent uses a ResNet-style actor–critic, `HexAlphaNet`. A 3×3 convolutional stem with batch normalisation lifts the two input planes to C channels, followed by B residual blocks, each two 3×3 convolutions with batch normalisation and a skip connection (**??**). Two heads share this trunk: a policy head (1×1 convolution to two planes, then a linear map to n^2 move logits) and a value head (1×1 convolution to one plane, a 256-unit hidden layer, and a $\tanh$ output in $[-1, +1]$). Invalid moves are masked by adding $-\infty$ to their logits before the softmax.

3.3 Search: PUCT with batched parallel evaluation

At each move the agent runs N MCTS simulations. Each simulation selects a path from the root by maximising the PUCT score

$$\mathrm{PUCT}(s,a) \;=\; -Q(s,a) \;+\; c_{\text{puct}}\, P(s,a)\, \frac{\sqrt{\sum_b N(s,b)}}{1+N(s,a)}, \tag{1}$$

where $N(s,a)$ is the child visit count, $P(s,a)$ the network prior, and $Q(s,a)$ the mean value stored *from the child's player's perspective*; the leading negation converts it to the selector's perspective, so we prefer children where the opponent fares worst. A leaf is expanded by one batched network evaluation that supplies priors for its children and a scalar value v for the leaf. Values are propagated to the root under the negamax convention, negating at each level:

$$W(s,a) \leftarrow W(s,a) + v, \qquad Q(s,a) = \frac{W(s,a)}{N(s,a)}, \qquad v \leftarrow -v. \tag{2}$$

Because a terminal Hex node always means the player to move has just lost, terminal leaves receive $v=-1$, which propagates correctly under Eq. (**??**).

To use a GPU efficiently we collect *several* leaves before each network call. During selection a *virtual loss* is added to every node on the in-flight path—raising its Q, lowering its $-Q$, and thus its PUCT score—so that subsequent concurrent selections diverge to different branches; the virtual loss is undone during backup (**?**). The number of leaves gathered per batch, p, must satisfy $p \le N/7$ so that at least seven sequential search rounds occur per move; otherwise the tree never grows beyond a couple of levels and the visit-count policy is nearly uniform. At the root we mix Dirichlet noise into the priors, $P \leftarrow (1-\varepsilon)P + \varepsilon\,\eta$ with $\eta \sim \mathrm{Dir}(\alpha)$, to ensure exploration. Self-play games batch this search across all concurrent games and, optionally, across multiple CPU worker processes, so a single network forward pass serves many positions at once.

A compiled C++ extension (`hex_mcts`) implements tree storage and traversal, exposing a batched `select_leaves`/`expand_and_backup` interface to Python; the pure-Python search is retained as a drop-in fallback. The C++ kernel removes per-node Python overhead, which dominates run time once the tree holds thousands of nodes.

3.4 Training objective

Self-play produces tuples (s,π,z) where π is the normalised root visit-count distribution and $z \in \{+1,-1\}$ records whether the player to move at s eventually won. The network is trained to match both targets:

$$\mathcal{L}(\theta) \;=\; \underbrace{-\textstyle\sum_a \pi(a)\,\log p_\theta(a \mid s)}_{\text{policy cross-entropy}} \;+\; \underbrace{\big(v_\theta(s)-z\big)^2}_{\text{value MSE}} \;+\; \lambda\,\|\theta\|_2^2, \tag{3}$$

with the L_2 term realised through decoupled weight decay in AdamW (**?**) rather than added to the loss explicitly. The learning rate follows a cosine schedule over the planned iterations. The boxed procedure in Figure **??** summarises one training iteration.

Algorithm 1: One AlphaZero training iteration for Hex.

Input: network θ, board size n, sims N, batch leaves p.

1. For each of $G(n)$ self-play games (batched together), until terminal:
   (a) Run N PUCT simulations (Eq. **??**), collecting p leaves per batched evaluation.
   (b) Set $\pi \leftarrow$ normalised root visit counts; store (s,π,player).
   (c) Sample a move from $\pi^{1/\tau}$ (τ-greedy after move T_{cut}) and advance.
   (d) Reuse the chosen child's sub-tree as the new root.
2. Label every stored s with $z=\pm 1$ from the outcome; augment by 180° symmetry.
3. Push samples to the replay buffer of capacity $R(n)$.
4. For E epochs: sample a mini-batch of size $\beta(n)$ and update θ by AdamW on Eq. **??**.
5. Step the cosine learning-rate schedule.

Figure 1: One AlphaZero training iteration for Hex.

3.5 Board-size scaling rules

The central design choice is that all capacity and budget quantities are functions of n. The per-move simulation budget targets roughly 3.5 visits per root child, rounded to a multiple of 50 and floored at 100:

$$N(n) \;=\; \max\Big(100,\; 50 \cdot \mathrm{round}\big(\tfrac{3.5\,n^2}{50}\big)\Big). \tag{4}$$

At play time the agent uses $2N(n)$ simulations, thinking harder against an opponent than during data generation. Network capacity steps up for larger boards—128 channels and 5 residual blocks for $n \le 9$, and 256 channels and 8 blocks for $n \ge 11$—because a deeper trunk is needed for the value head's receptive field to span full-board connectivity. The number of self-play games per iteration $G(n)$, the replay capacity $R(n)$ (sized to hold roughly seventy iterations of data), and the mini-batch size $\beta(n)$ likewise grow with n. Table **??** lists the resulting schedule.

Table 1: Board-size scaling schedule. Simulation budget from Eq. (**??**); inference budget is twice the self-play budget. Network sizes and self-play volume step up at $n \ge 11$.

Board n	Self-play sims N	Inference sims	Net (ch×blk)	Games/iter	Params (M)
7	150	300	128×5	16	—
9	300	600	128×5	16	—
11	400	800	256×8	16	9.51
13	600	1200	256×8	24	9.55
15	800	1600	256×8	32	9.61
17	1000	2000	256×8	32	9.69

Design rationale. The constant 3.5 in Eq. (**??**) is the load-bearing choice. With far fewer visits per child, most children of the root receive zero or one visit and the visit-count policy π degenerates into noise, so the cross-entropy target in Eq. (**??**) teaches the network nothing and the bootstrap stalls—the failure mode we observed when a fixed small budget was reused on large boards. A much larger constant would make self-play prohibitively slow. Tying capacity, data volume, and search budget together with a single input also removes the practitioner's most common error: forgetting to retune one of them when changing n.

3.6 Practical refinements

Five additions, each standard in strong implementations, complete the system. **(1) Symmetry augmentation:** in the perspective frame the rhombus has one non-trivial symmetry, a 180° rotation that maps Red's goal to itself; every sample is duplicated with its rotated state and policy, doubling data for free. **(2) Sub-tree reuse:** after a move, the chosen child's sub-tree is kept as the new root rather than rebuilt, roughly halving effective search cost in deep positions. **(3) Calibrated resignation:** when the root value stays below -0.95 for several consecutive moves the game is conceded to save compute, while a 10% fraction of games never resign so value targets stay calibrated. **(4) Mixed precision:** `bfloat16` autocast accelerates GPU training and inference with no loss scaler, since bf16 shares fp32's exponent range. **(5) AdamW with cosine annealing** replaces plain Adam plus a manual L_2 term, decoupling weight decay from the adaptive step.

4 Experiments

Setup. All training was performed from scratch on a single CUDA GPU using the C++ MCTS extension. We train independent agents per board size; results below cover 11×11, 13×13, and 15×15, with longer runs configured for 15×15 and 17×17. The opponent for periodic evaluation is a uniform-random player: across 20 games (alternating colours) the agent selects moves greedily by visit count using a reduced budget of $\min(N, 15)$ simulations, a deliberately conservative probe that is cheap enough to run every 20 iterations. We report the win rate against random and the two loss components from Eq. (**??**). Table **??** lists the fixed hyperparameters; size-dependent quantities follow Table **??**.

Table 2: Fixed training and search hyperparameters (size-independent).

Hyperparameter	Value	Hyperparameter	Value
PUCT constant c_{puct}	1.5	Learning rate (initial)	10^{-3}
Virtual loss	1	Weight decay λ	10^{-4}
Dirichlet α	0.3	Gradient clip (norm)	1.0
Dirichlet mix ε	0.25	Train epochs/iter E	5
Temperature τ	1.0	Optimizer	AdamW
Temperature cutoff T_{cut}	10 moves	LR schedule	cosine

Compute. All experiments were run on a single GPU. Representative throughput: the 11×11 and 13×13 agents complete 150 iterations (about $2{,}400$ self-play games) in a few hours each; the auto-scaled 13×13 run extends to 250 iterations ($\approx 5{,}040$ games) at 600 simulations per move.

5 Results

Learning against a random opponent. Table **??** reports the win rate versus the random opponent at evaluation checkpoints. On 11×11 the agent rises from chance (50%) at iteration 20 to 90% by iteration 40 and reaches 100% by iteration 80, fluctuating between 95% and 100% thereafter. The 13×13 agent learns comparably fast, already at 85% by iteration 20 and saturating near 100%. The 15×15 agent, with its larger branching factor, climbs more gradually—from 55% at iteration 20 to 90% by iteration 120—consistent with the expectation that larger boards need more iterations even with the scaled budget.

Table 3: Win rate (%) versus a uniform-random opponent over 20 games, measured every 20 training iterations with a reduced evaluation budget. Higher is better; 50% is chance.

Board	it. 20	it. 40	it. 60	it. 80	it. 100	it. 120	it. 140
11×11	50	90	90	**100**	95	**100**	95
13×13	85	90	90	**100**	95	95	**100**
15×15	55	75	75	70	85	90	80

Extended self-play at higher simulation budgets. Resuming the 13×13 agent under the auto-scaled budget (600 simulations per move, 24 games per iteration) and training to iteration 240 holds the win rate at 95–100% from iteration 60 onward (e.g. 95% at it. 60, 100% at it. 140, 100% at it. 240), confirming that the higher-budget regime sustains rather than destabilises the learned policy.

Loss curves. The value-head mean-squared error settles into the 0.4–0.6 range early and remains stable, indicating calibrated outcome predictions. The policy cross-entropy decreases slowly—on 13×13 from about 4.5 at iteration 20 to 3.9 at iteration 240—which is expected: the target π is itself a moving, search- generated distribution over n^2 moves, so its entropy floor keeps the absolute loss high even as the policy sharpens. Table **??** summarises representative values.

Table 4: Representative loss components from Eq. (**??**) at selected iterations. Policy loss is cross-entropy against the MCTS visit distribution; value loss is MSE against game outcome.

Board (run)	Iteration	Policy loss	Value loss	Win%
11×11	20	4.19	0.59	50
11×11	140	3.98	0.61	95
13×13	20	4.51	0.51	85
13×13	140	4.14	0.45	100
13×13 (600 sims)	240	3.92	0.49	100
15×15	20	4.76	0.43	55
15×15	140	4.23	0.50	80

Effect of the scaling rules. The qualitative effect of Section **??** is decisive: with a fixed small simulation budget reused on $\geq 11 \times 11$ boards, the visit-count policy target is near-uniform and the agent does not learn, remaining at chance against random play. Applying Eq. (**??**) restores the rapid learning seen in Table **??**.

6 Discussion

6.1 Interpretation

The results support the paper's thesis that, for knowledge-free AlphaZero on Hex, the governing variable is whether search is deep enough to make the visit-count policy informative, and that this can be guaranteed by tying the simulation budget to n^2. The ordering of learning speed across boards—fastest on 11×11 and 13×13, slower on 15×15—matches the prediction that a larger action space dilutes a fixed number of visits and demands more iterations. That the extended 13×13 run remains stable at a four-fold higher simulation budget indicates the loop is robust to the budget once the minimum-visit threshold is met, rather than finely balanced on it. The stable, low value loss alongside a high but slowly falling policy loss is the signature of a healthy AlphaZero run on a large action space: outcome prediction is easy to calibrate, whereas the policy chases a non-stationary, high-entropy target.

6.2 Practical implications

A single integer suffices to configure the agent for any board in the supported range, which is precisely what a course or library setting needs: one code path, no per-size retuning, and a documented failure mode (too-shallow search) with a documented remedy. The perspective frame and symmetry augmentation show how a small amount of game-structure awareness—one canonical orientation and one rotation—reduces both the learning problem and the data cost without importing Hex-specific strategy. The compiled search kernel and batched, multi-process self-play illustrate that the throughput bottleneck for small-net AlphaZero is tree-traversal overhead and GPU under-utilisation, both addressable without changing the algorithm.

6.3 Relation to prior work

Our win rates are reported only against a random opponent, whereas specialist Hex engines such as MoHex are evaluated against each other and against humans (**??**). Defeating random play is a necessary but weak milestone; it confirms the pipeline learns and is stable, but says little about absolute strength. The clear next comparison is against the value-based and policy-gradient baselines in the same repository (**??**) and, ultimately, against a knowledge-rich solver. We expect the search-based agent to dominate the model-free baselines—this is the standard finding since ExIt (**?**)—and to trail knowledge-rich engines until Hex-specific inferior-cell and virtual-connection reasoning is added.

Limitations. Three limitations frame future work. First, evaluation is only against a random opponent; head-to-head matches against the model-free baselines and a strong reference engine are needed to quantify absolute strength. Second, the longest runs for 15×15 and 17×17 are configured but not yet reported to convergence here, so conclusions for the largest boards are preliminary. Third, the agent uses no Hex-specific knowledge, no Gumbel or playout-cap search improvements, and no resignation ablation; each is a concrete, additive opportunity rather than a structural obstacle.

7 Conclusion

We presented a complete, knowledge-free AlphaZero agent for Hex whose defining feature is that every search and learning quantity—simulation budget, network width and depth, self-play volume, replay capacity, and batch size—is derived automatically from the board size through explicit rules. The motivating insight is that learning bootstraps only when the search visits each root child often enough to make the visit-count policy informative, a condition that fails silently when a small-board budget is reused on a large board and is restored by scaling the budget with n^2. Around this core we built a reproducible single-GPU system with a colour-agnostic perspective frame, a compiled C++

MCTS engine with virtual-loss parallel leaf evaluation, pipelined multi-process self-play, and five standard refinements. Trained from scratch, the agent reaches 90–100% win rates against a random opponent on 11×11 and 13×13 within tens of iterations and continues to improve on 15×15.

Three concrete directions follow. First, replace the random-opponent metric with head-to-head Elo against the repository's DQN, PPO, and REINFORCE baselines and against a reference Hex engine, to measure absolute rather than relative strength. Second, run the 15×15 and 17×17 agents to convergence and conduct a paired, compute-matched ablation of the scaling rules and each of the five refinements. Third, add modern search and knowledge improvements—Gumbel-AlphaZero action selection, playout caps, and Hex-specific inferior-cell pruning—to close the gap to specialist engines. More broadly, the size-scaling principle suggests a path toward agents that self-configure across a family of related tasks from a single structural parameter, reducing the manual tuning that makes self-play reinforcement learning fragile.

Acknowledgments and Disclosure of Funding

This work was developed as part of the reinforcement learning coursework at the University of Applied Sciences Technikum Wien (FHTW). The author declares no competing interests and received no external funding.

References

References

Nash, J. (1952). Some games and machines for playing them. *RAND Corporation Technical Report*, D-1164.

Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., van den Driessche, G., et al. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587), 484–489.

Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., et al. (2017). Mastering the game of Go without human knowledge. *Nature*, 550(7676), 354–359.

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., et al. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419), 1140–1144.

Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., et al. (2020). Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588(7839), 604–609.

Anthony, T., Tian, Z., & Barber, D. (2017). Thinking fast and slow with deep learning and tree search. *Advances in Neural Information Processing Systems*, 30.

Gao, C., Hayward, R., & Müller, M. (2017). Move prediction using deep convolutional neural networks in Hex. *IEEE Transactions on Games*, 10(4), 336–343.

Hayward, R.B., & Toft, B. (2019). *Hex: The Full Story*. CRC Press.

Kocsis, L., & Szepesvári, C. (2006). Bandit based Monte-Carlo planning. *European Conference on Machine Learning (ECML)*, 282–293.

Browne, C., Powley, E., Whitehouse, D., Lucas, S., Cowling, P.I., Rohlfshagen, P., et al. (2012). A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1), 1–43.

Chaslot, G.M.J.-B., Winands, M.H.M., & van den Herik, H.J. (2008). Parallel Monte-Carlo tree search. *International Conference on Computers and Games*, 60–71.

Tian, Y., Ma, J., Gong, Q., Sengupta, S., Chen, Z., Pinkerton, J., & Zitnick, C.L. (2019). ELF OpenGo: An analysis and open reimplementation of AlphaZero. *International Conference on Machine Learning (ICML)*, 6244–6253.

Wu, D.J. (2019). Accelerating self-play learning in Go. *arXiv preprint* arXiv:1902.10565.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *International Conference on Machine Learning (ICML)*, 448–456.

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR)*.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint* arXiv:1707.06347.

Williams, R.J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3), 229–256.

Watkins, C.J.C.H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3), 279–292.

A Implementation notes

The agent interface is a single callable `agent(board, action_set)` returning a (row, col) move. On the first move of an empty board the agent plays a random interior cell, avoiding the outer ring whose corner/edge openings are weak in Hex; all subsequent moves use the full inference-time MCTS budget $2N(n)$. Models are persisted per board size with their channel and block counts so they reload with the correct architecture. Self-play workers run network inference on CPU to avoid contending with the GPU training process, communicating weights and samples through multiprocessing queues with the file-system tensor-sharing strategy to avoid file-descriptor exhaustion. The repository also contains model-free baselines—tabular Q-learning, DQN (with an optional minimax variant), REINFORCE, and PPO—behind the same interface for comparison.

B Reproducibility

Training is launched via `python train.py -method alphazero -size N -iterations K`, with all size-dependent quantities defaulting to the rules of Section **??** unless overridden. A guard warns when the requested parallel leaf count exceeds $N/7$, which would make the search too shallow to learn. Checkpoints are written every 20 iterations, enabling resumption at a higher simulation budget as used for the extended 13×13 run.